

Manual

This document describes the current behavior of [benchmark/retrieval_config_tuning.py](#).

Trials run the shared retrieval path through `get_answer(..., run_mode="retrieval_only")`. Tuning therefore measures retrieval quality directly, without paying final answer-generation cost on every trial.

Prerequisites

The tuning script supports both online and offline model backends, but the current implementation still expects several environment variables to be present at import time.

Before running the tuning script, make sure the project `.env` file includes:

```
GOOGLE_API_KEY="your_google_api_key"
CLOUDFLARE_ACCOUNT_ID="your_cloudflare_account_id"
CLOUDFLARE_AUTH_TOKEN="your_cloudflare_auth_token"
OPENROUTER_API_KEY="your_openrouter_api_key"
```

These variables are used as follows:

- `GOOGLE_API_KEY` : required by the default chat model `gemini-3.1-flash-lite`. The tuning script uses this chat model to precompute or refresh the BM25 query rewrite cache in hybrid retrieval mode.
- `CLOUDFLARE_ACCOUNT_ID` and `CLOUDFLARE_AUTH_TOKEN` : used by the online embedding registry entry `Qwen3-Embedding-0.6B-4bit-DWQ-Online` in [src/powerllm/retrieval/pipeline.py](#). In the current code, these values are validated when that module is imported, so they must be present even if the selected tuning run uses the local default embedding model.
- `OPENROUTER_API_KEY` : loaded at import time in [src/powerllm/models/model_resolver.py](#) because the model registry includes `qwen-3.5-9b-openrouter`. In the current implementation, this variable must also be present even though the default tuning configuration does not use OpenRouter.

The default embedding model for tuning is:

```
Qwen3-Embedding-0.6B-4bit-DWQ
```

This model is configured as an OpenAI-compatible local endpoint at:

```
http://127.0.0.1:1234/v1
```

So, besides populating `.env`, make sure the local embedding server is running and serving that model before starting the tuning job.

To change the embedding model options used by tuning, edit the embedding registry in [pipeline.py](#).

To change the chat model options used by tuning, edit the chat model registry in [model_resolver.py](#).

Quick Start

```
# Install Optuna if needed
venv/bin/pip install optuna

# Budget-aware TPE search on the mixed tuning dataset
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --search-method tpe \
  --study-name fixed_chunking_2000_tpe_demo \
  --n-trials 50 \
  --limit-contracts 10 \
  --limit-questions 5

# Budget-aware random search
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --search-method random \
  --study-name fixed_chunking_2000_random_demo \
  --n-trials 50 \
  --limit-contracts 10 \
  --limit-questions 5

# No-budget empirical grid baseline
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space empirical_grid_baseline \
  --search-method grid \
  --disable-budget-pruning \
  --study-name empirical_grid_baseline_demo \
  --n-trials 108 \
```

```
--limit-contracts 10 \  
--limit-questions 5
```

Keep tuning runs sequential. The script exposes `--n-jobs`, but benchmark and tuning runs in this project should still be executed one at a time because they share caches and output locations. In practice, keep `--n-jobs 1`.

Current CLI

The current CLI requires both `--study-name` and `--search-space`.

Required arguments

- `--study-name` : Optuna study name
- `--search-space` : search-space preset name

Common optional arguments

- `--n-trials`
- `--storage`
- `--limit-contracts`
- `--limit-questions`
- `--dataset`
- `--search-method` (alias: `--search_method`)
- `--disable-budget-pruning`
- `--density-precision-target`
- `--n-jobs`
- `--seed`
- `--embedding-model-id`
- `--chunking-strategy` (`legal` or `langchain_basic`)
- `--output`
- `--use-reranker`
- `--retry-zero-trials`

Current Defaults

- Dataset: `benchmark/legalbench_mixed_config_q200_c20_seed42.json`

- Output directory: `benchmark_results/tuning`
- Optuna storage: `sqlite:///benchmark/config_tuning.db`
- Retrieval mode: `hybrid`
- Embedding model: `Qwen3-Embedding-0.6B-4bit-DWQ`
- Chat model: `gemini-3.1-flash-lite`
- BM25 rewrite mode: `concurrent`
- BM25 rewrite cache `k`: `50`
- Chunking strategy: `legal`
- Default contracts per trial: `4`
- Default questions per contract: `5`
- Default search method: `tpe`
- Budget pruning: enabled by default
- Final context budget: `22,000` characters
- Reranker: disabled unless `--use-reranker` is passed

Search-Space Presets

The script currently exposes the following presets through `--search-space`.

`fixed_chunking_2000`

- `chunk_size = 2000`
- `chunk_overlap = 400`
- `similarity_k = 4..16`
- `bm25_k = 20..100`
- `final_k = 4..16`
- `rrf_k = 60`

This is the main budget-aware retrieval-only preset used in recent experiments.

`fixed_chunking_1000`

- `chunk_size = 1000`
- `chunk_overlap = 200`
- `similarity_k = 4..16`
- `bm25_k = 20..100`
- `final_k = 4..16`
- `rrf_k = 60`

fixed_chunking_1500

- `chunk_size = 1500`
- `chunk_overlap = 300`
- `similarity_k = 4..16`
- `bm25_k = 20..100`
- `final_k = 4..16`
- `rrf_k = 60`

empirical_grid_baseline

- `chunk_size ∈ {512, 1000, 2000}`
- `chunk_overlap_ratio = 0.20` (fixed, so overlap is 20% of chunk size)
- `similarity_k ∈ {4, 8, 12}`
- `bm25_k ∈ {40, 60, 80}`
- `final_k ∈ {8, 10, 12, 14}`
- `rrf_k = 60`

This preset is intended for the manual no-budget grid baseline. In practice it is usually paired with:

```
--search-method grid --disable-budget-pruning
```

baseline_kset

- `chunk_size ∈ {256, 512, 1000, 1500, 2000, 2100}`
- `chunk_overlap_ratio = 0.20` (fixed)
- `k_set ∈ {A, B, C}`
- fixed retrieval defaults outside the selected set:
 - `similarity_k = 9`
 - `bm25_k = 39`
 - `final_k = 10`
 - `rrf_k = 60`

`k_set` maps to:

- A: `similarity_k=8, bm25_k=61`
- B: `similarity_k=7, bm25_k=49`
- C: `similarity_k=13, bm25_k=81`

tune_chunking_and_final_k

- `chunk_size` ∈ {256, 512, 1000, 1500, 2000, 2100}
- `chunk_overlap_ratio` = 0.20 (fixed)
- `similarity_k` = 9 (fixed)
- `bm25_k` = 39 (fixed)
- `final_k` = 4..16
- `rrf_k` = 60 (fixed)

tpe_wide

- `chunk_size` = 2000..9000 in steps of 250
- `chunk_overlap_ratio` = 0.10..0.35 in steps of 0.05
- `similarity_k` = 4..24
- `bm25_k` = 20..140 in steps of 5
- `rrf_k` = 20..120 in steps of 10
- `final_k` = 10 (fixed)

small_chunks

- `chunk_size` = 500..3000 in steps of 250
- `chunk_overlap_ratio` = 0.10..0.35 in steps of 0.05
- `similarity_k` = 4..24
- `bm25_k` = 20..140 in steps of 5
- `rrf_k` = 20..120 in steps of 10
- `final_k` = 10 (fixed)

Search Families

Presets are internally grouped into three search families:

- `retrieval` : fixed chunking, tune retrieval parameters
- `joint` : tune chunking and retrieval parameters together
- `chunking` : tune chunking while keeping retrieval mostly fixed

For current presets:

- `fixed_chunking_2000`, `fixed_chunking_1000`, `fixed_chunking_1500` -> `retrieval`

- `empirical_grid_baseline`, `baseline_kset`, `tpe_wide`, `small_chunks` -> `joint`
- `tune_chunking_and_final_k` -> `chunking`

Budget Pruning

Budget pruning is enabled by default. The script estimates final context size as:

```
estimated_context_chars = chunk_size * final_k
```

Trials that exceed the fixed budget of `22,000` characters are pruned unless `--disable-budget-pruning` is passed.

This is especially important for comparing:

- budget-aware `tpe` or `random` search
- no-budget manual `grid` baselines

Dataset Shape

The script expects a LegalBench-RAG style JSON dataset with a top-level `tests` array. It converts that file into grouped contract-level records:

- each contract record contains `title`, full `context`, and a list of `qas`
- each QA item contains `id`, `question`, `answers`, and `is_impossible`
- answers are converted into span annotations of the form `{answer_start, text}`

Contracts are loaded from `corpus/<file_path>` next to the dataset JSON. `--limit-contracts` and `--limit-questions` are applied after grouping.

Metrics Used During Tuning

The tuning script imports its retrieval metrics from [benchmark/benchmark_metrics.py](#).

Current metric cutoffs come from the shared defaults:

- `recall_ks = (10, 20)`
- `hit_ks = (1, 3, 5)`
- `precision_ks = (5, 10, 20)`

Per-question metric definitions

- `recall@k` : fraction of valid gold answer spans fully covered by the top- `k` retrieved chunks
- `recall@final_k` : fraction of valid gold answer spans fully covered by the final retrieved window
- `hit@k` : `1.0` if any valid gold span is fully covered within top- `k` , else `0.0`
- `precision@k` : fraction of top- `k` retrieved chunks that fully cover at least one valid gold answer span
- `precision@final_k` : the same precision computed on the final retrieved window
- `f1@k` : harmonic mean of recall and precision when both are defined
- `MRR` : reciprocal rank of the first retrieved chunk that fully covers any valid gold span
- `fully_covered_at_final_k` : `1.0` when all valid gold spans are covered in the final window, else `0.0`
- `mean_overlap_ratio_at_final_k` : for each gold span, take the best overlap ratio against the final retrieved chunks, then average those span-level ratios

Questions without valid gold spans are excluded from retrieval metric means and counted under `no_answer_questions`.

Coverage labels

Each answerable question is classified by final-window coverage:

- `Yes` : all valid spans covered
- `partial` : some but not all valid spans covered
- `No` : no valid spans covered

Aggregated coverage reports:

- `fully_covered`
- `partial_covered`
- `not_covered`
- `partial_or_better`

All coverage rates are normalized by `answerable_questions`.

Objective Function

Single-objective tuning optimizes a weighted `quality_score`.


```
precision_at_final_k = metrics["metrics"]["precision"]["at_final_k_mean"]
mean_overlap_ratio_at_final_k = (
    metrics["metrics"]["context_quality"]["mean_overlap_ratio_at_final_k_
)

quality_score = (
    0.30 * recall_at_final_k
    + 0.18 * partial_or_better_rate
    + 0.22 * fully_covered_at_final_k
    + 0.15 * mrr_mean
    + 0.10 * hit_at_5
    + 0.05 * mean_overlap_ratio_at_final_k
)
```

`density_score` and `precision_at_final_k` are still recorded in the output for analysis, but they are not part of the current optimization objective.

Latency is recorded, exported, and printed, but it is not part of the single-objective score.

Pruning and Deduplication

- **TPE search** uses `MedianPruner` with `n_startup_trials=5`. Trials can be pruned after their first intermediate report.
- **Grid and random search** use `NopPruner` so every sampled configuration runs to completion and gets a comparable score.
- **Trial deduplication**: the objective function caches evaluated parameter combinations. If Optuna samples the exact same parameters again, the cached score is returned immediately without re-running the benchmark.

BM25 Query Cache Behavior

For `bm25` and `hybrid` tuning, BM25 rewrites are precomputed or reused from:

- `benchmark/query_cache/bm25_query_cache_{dataset_stem}_{CHAT_MODEL_ID}_k50.json`

Behavior:

- the cache is keyed by `(title, question_id, question)`
- each trial reads `bm25_query` from that cache

- the cached rewrite is passed as `bm25_query_override`
- trial-level `bm25_k` still controls how many BM25 candidates are retrieved during evaluation

Runtime Behavior

- embedding model connectivity is always checked before tuning
- explicit chat model connectivity is only checked when `RETRIEVAL_MODE != "hybrid"`
- in the default hybrid flow, the chat model may still be needed to populate the BM25 rewrite cache if it does not already exist
- trials reuse persistent vector caches for matching chunking configurations
- zero-score trials can be re-enqueued with `--retry-zero-trials` (useful for recovering from transient failures)

Common Commands

Budget-aware TPE search

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --search-method tpe \
  --study-name fixed_chunking_2000_tpe_q50_c10x5 \
  --n-trials 100 \
  --limit-contracts 10 \
  --limit-questions 5
```

Budget-aware random search

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --search-method random \
  --study-name fixed_chunking_2000_random_q50_c10x5 \
  --n-trials 100 \
  --limit-contracts 10 \
  --limit-questions 5
```

No-budget empirical grid baseline

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space empirical_grid_baseline \
  --search-method grid \
  --disable-budget-pruning \
  --study-name empirical_grid_baseline_no_budget_q50_c10x5 \
  --n-trials 108 \
  --limit-contracts 10 \
  --limit-questions 5
```

Resume an interrupted study

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --study-name my_tuning \
  --n-trials 200 \
  --storage sqlite:///benchmark/config_tuning.db
```

Retry zero-score trials

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --study-name my_tuning \
  --retry-zero-trials \
  --n-trials 10 \
  --storage sqlite:///benchmark/config_tuning.db
```

Custom dataset

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --study-name custom_dataset_run \
  --dataset benchmark/my_custom_dataset.json \
  --n-trials 50
```

Custom output path

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --study-name custom_output_run \
  --n-trials 50 \
  --output benchmark_results/tuning/my_experiment.json
```

Enable reranker

```
venv/bin/python -m benchmark.retrieval_config_tuning \
  --search-space fixed_chunking_2000 \
  --study-name reranker_run \
  --use-reranker \
  --n-trials 50
```

Output JSON

Runs write `best_trial` plus `summary`.

Top-level shape

```
{
  "best_trial": {
    "trial_number": 72,
    "score": 0.5211,
    "params": {
      "chunk_size": 2000,
      "chunk_overlap": 400,
      "similarity_k": 10,
      "bm25_k": 41,
      "final_k": 11,
      "rrf_k": 60
    },
    "metrics": {},
    "coverage": {},
    "failure_analysis": {},
    "quality_score": 0.5211,
    "density_score": 1.0,
    "density_precision_target": 0.05,
    "precision_at_final_k": 0.0836,
```

```
    "avg_latency": 0.0756,
    "max_latency": 0.1200
},
"summary": {
  "run": {
    "status": "completed",
    "started_at": "2026-04-25T00:00:00+00:00",
    "completed_at": "2026-04-25T00:10:00+00:00",
    "output_path": "benchmark_results/tuning/best_params_20260425_000000"
  },
  "dataset": {
    "dataset_path": "benchmark/legalbench_mixed_config_q200_c20_seed42.",
    "contracts_evaluated": 10,
    "questions_evaluated": 50,
    "answerable_questions": 50,
    "no_answer_questions": 0
  },
  "config": {
    "embedding_model_id": "Qwen3-Embedding-0.6B-4bit-DWQ",
    "chat_model_id": "gemini-3.1-flash-lite",
    "retrieval_mode": "hybrid",
    "chunking_strategy": "legal",
    "objective": {
      "density_precision_target": 0.05
    },
    "retrieval_params": {
      "similarity_k": 10,
      "bm25_k": 41,
      "final_k": 11,
      "rrf_k": 60
    },
    "metric_k_config": {
      "recall_ks": [10, 20],
      "hit_ks": [1, 3, 5],
      "precision_ks": [5, 10, 20]
    },
    "chunking": {
      "chunk_size": 2000,
      "chunk_overlap": 400
    },
    "query_rewrite": {
      "llm_mode": "concurrent",
      "bm25_query_cache_k": 50,
      "bm25_query_cache_path": "benchmark/query_cache/..."
    }
  }
}
```

```

    },
    "storage": {
        "output_dir": "benchmark_results/tuning",
        "vector_cache_dir": "benchmark/chroma_cache"
    }
},
"tuning": {
    "search_family": "retrieval",
    "search_method": "tpe",
    "search_space": "fixed_chunking_2000",
    "trials_requested": 100,
    "trials_completed": 100,
    "best_trial_number": 72,
    "best_score": 0.5211,
    "budget_pruning_enabled": true,
    "context_char_budget": 22000
},
"metrics": {
    "recall": {},
    "hit": {},
    "precision": {},
    "f1": {},
    "ranking": {},
    "context_quality": {},
    "latency": {
        "avg_seconds": 0.0756,
        "max_seconds": 0.1200
    },
    "context_cost": {
        "avg_context_chars": 21500.0
    }
},
"coverage": {
    "fully_covered": {},
    "partial_covered": {},
    "not_covered": {},
    "partial_or_better": {}
},
"failure_analysis": {}
}
}

```

Notes:

- `best_trial` stores the selected trial, including the resolved full parameter set, the aggregated retrieval metrics for that trial, and key diagnostics such as `quality_score` , `density_score` , and latency.
- `summary.run` records run status, timestamps, and the output file path.
- `summary.dataset` records which dataset was evaluated and how many contracts/questions were included.
- `summary.config` records the resolved embedding model, chat model, retrieval mode, chunking strategy, retrieval parameters, metric cutoffs, BM25 cache settings, and output/cache locations.
- `summary.tuning` records the search family, search method, preset name, requested/completed trial counts, best trial number, best score, and budget-pruning status.
- `summary.metrics` includes the aggregated retrieval metrics and diagnostics:
 - `recall`
 - `hit`
 - `precision`
 - `f1`
 - `ranking`
 - `context_quality`
 - `latency ( avg_seconds , max_seconds )`
 - `context_cost ( avg_context_chars )`
- `summary.coverage` stores final-window evidence coverage rates:
 - `fully_covered`
 - `partial_covered`
 - `not_covered`
 - `partial_or_better`
- `summary.failure_analysis` stores miss diagnostics exported from the shared benchmark metrics pipeline.

Inspecting Results

The script prints the storage URL at the end. To inspect a study:

```
optuna-dashboard sqlite:///benchmark/config_tuning.db
```